Software Architecture and Design Specification

Project: Operation Theatre Management System (OTMS)
Version: 1.0
Authors: PES1UG24AM261, PES1UG24AM281, PES1UG24AM289, PES1UG24AM295
Date: 06-10-2026
Status: Vers 1

Revision History

Version	Date	Authors	Change Summary
1.0	06-10-2026	PES1UG24AM261, PES1UG24AM281, PES1UG24AM289, PES1UG24AM295	Initial SAD document

Approvals

Role	Name	Signature/Date
Course Coordinator	<Name>	<Signature/Date>
Project Guide	<Name>	<Signature/Date>

---

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Operation Theatre Management System (OTMS). It describes the major software components, their interactions, architectural decisions, security architecture, APIs, and system design.

1.2 Scope

The system manages operation theatre scheduling, surgeries, doctors, nurses, equipment, theatre availability, emergency cases, and resource allocation. It detects scheduling conflicts and provides information to improve operation theatre and resource utilization.

1.3 Audience

The intended audience includes developers, QA engineers, hospital administrators, doctors, operation theatre coordinators, security auditors, maintenance teams, and instructors.

1.4 Definitions

- OTMS – Operation Theatre Management System
- OT – Operation Theatre
- Surgery – A planned or emergency surgical procedure
- OR – Operation Room / Operation Theatre
- RBAC – Role-Based Access Control
- API – Application Programming Interface
- UI – User Interface
- DB – Database
- ADR – Architecture Decision Record
- TLS – Transport Layer Security

---

2. Document Overview

2.1 How to Use This Document

This document provides the architectural and design specifications of OTMS, including UML diagrams, component descriptions, architectural decisions, security architecture, API design, sequence diagrams, error handling, and monitoring mechanisms.

2.2 Related Documents

- Software Requirements Specification (SRS)
- Software Test Plan (STP)
- Requirements Traceability Matrix (RTM)

---

3. Architecture

3.1 Goals & Constraints

Goals

- Provide reliable and efficient operation theatre scheduling.
- Detect doctor, nurse, equipment, and theatre scheduling conflicts.
- Prioritize emergency surgeries appropriately.
- Improve operation theatre utilization.
- Provide secure access based on user roles.
- Maintain accurate surgery and resource information.
- Provide timely notifications regarding schedule changes and conflicts.

Constraints

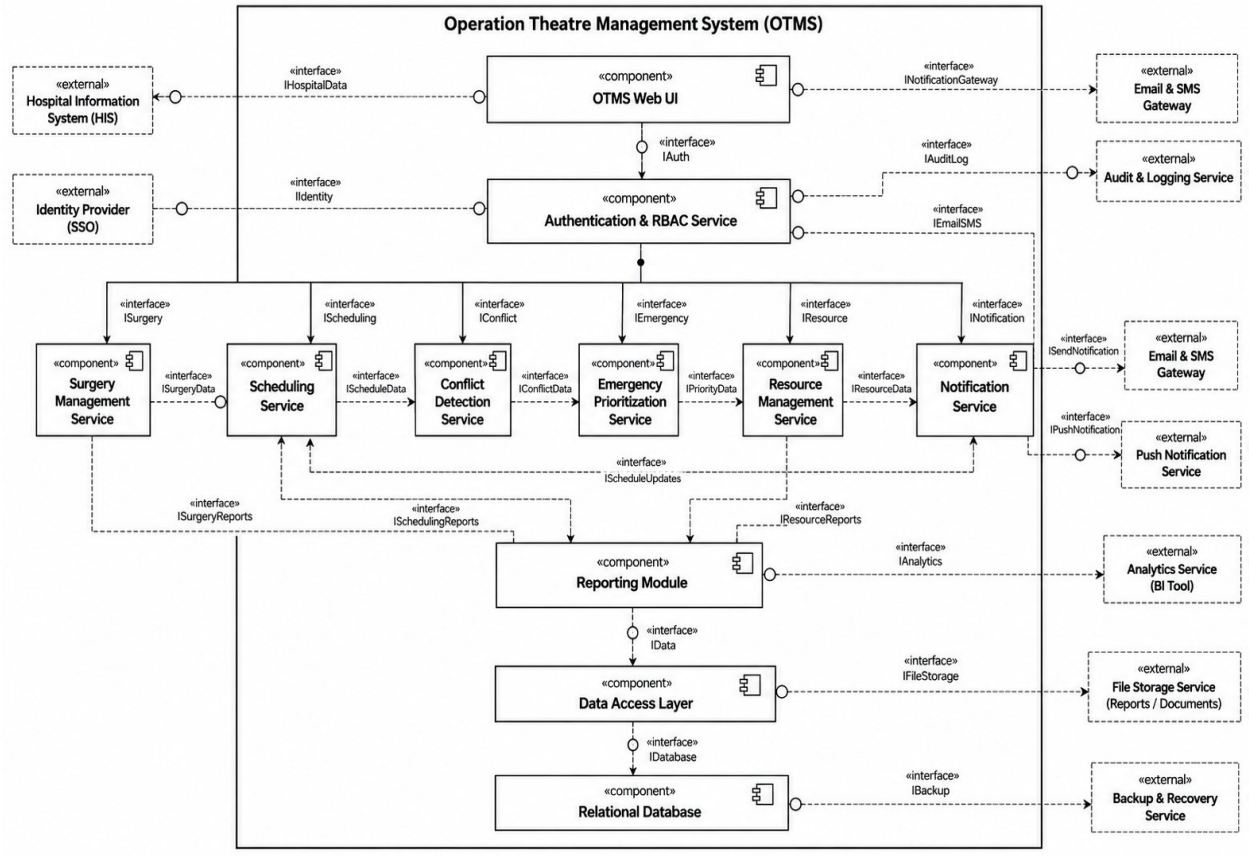
- Patient and surgery information must be protected.
- The system must support multiple hospital users simultaneously.
- Scheduling must prevent double-booking of theatres and critical resources.
- Emergency cases must be accommodated without compromising existing schedules unnecessarily.
- The system depends on accurate availability information from hospital staff.
- Access to sensitive patient information must be restricted according to user roles.

3.2 Stakeholders & Concerns

Stakeholder	Main Concerns
Hospital Administrator	Overall operation theatre utilization, reports, system management
OT Coordinator	Scheduling, conflict detection, theatre availability
Doctors/Surgeons	Surgery schedules, availability, assigned theatres
Nurses	Assigned surgeries, schedules, required equipment
Anaesthesiologists	Surgery schedule, patient and procedure information
Technicians	Equipment availability and maintenance

Hospital Management	Resource utilization and performance reports
Developers	Maintainability, modularity, scalability
QA Engineers	Testability, reliability, correctness
Security/Admin Team	Authentication, authorization, data protection

3.3 Component (UML) Diagram



3.4 Component Descriptions

OTMS UI

Provides interfaces for hospital administrators, OT coordinators, doctors, nurses, and technicians. It allows users to view schedules, manage resources, and receive conflict notifications.

Authentication & RBAC Service

Authenticates users and controls access based on roles such as administrator, OT coordinator, doctor, nurse, and technician.

Surgery Management Service

Creates and manages surgery records, including surgery type, assigned doctor, required staff, estimated duration, priority, and status.

Scheduling & Conflict Detection Service

Assigns surgeries to available theatre slots and detects conflicts involving operation theatres, doctors, nurses, and equipment.

Resource Management Service

Maintains availability and allocation information for operation theatres, doctors, nurses, anaesthesia resources, and surgical equipment.

Emergency Prioritization Service

Identifies emergency surgeries and assigns them higher scheduling priority while checking the impact on already scheduled surgeries.

Notification Service

Sends notifications to relevant users when surgeries are scheduled, rescheduled, cancelled, or affected by conflicts or emergencies.

Database

Stores information about patients, surgeries, doctors, nurses, theatres, equipment, schedules, conflicts, and audit records.

Hospital Systems/API

Provides integration with other hospital systems where required, such as patient records or staff information systems.

3.5 Chosen Architecture Pattern and Rationale

A layered architecture is chosen for OTMS because it provides clear separation between the user interface, business logic, scheduling logic, and data storage.

The system can be divided into:

1. Presentation Layer – OTMS web interface and dashboards.
2. Application/Service Layer – Surgery, scheduling, resource, notification, and authentication services.
3. Business Logic Layer – Conflict detection, emergency prioritization, scheduling rules, and resource allocation.
4. Data Access Layer – Handles communication with the database.
5. Database Layer – Stores persistent system data.

A fully distributed microservices architecture is not selected because it would introduce unnecessary deployment and communication complexity for the initial hospital system.

3.6 Technology Stack & Data Stores

Area	Technology
Frontend	HTML, CSS, JavaScript / React
Backend	Java with Spring Boot
API	REST APIs
Database	MySQL / PostgreSQL
Authentication	JWT / secure session-based authentication

Communication Security	TLS
Diagram Tools	PlantUML / draw.io
API Testing	Postman
Version Control	Git

Data Stores

The primary relational database stores:

- Patient information
- Surgery details
- Doctor information
- Nurse information
- Theatre information
- Equipment information
- Theatre schedules
- Resource allocations
- Scheduling conflicts
- Emergency cases
- Audit logs

3.7 Risks & Mitigations

Risk	Mitigation
Two surgeries being assigned to the same theatre slot	Automated conflict detection before confirmation
Doctor being assigned to overlapping surgeries	Validate doctor availability during scheduling
Required equipment being unavailable	Check equipment availability before scheduling

Emergency surgery disrupting existing schedule	Emergency prioritization and rescheduling mechanism
Incorrect resource availability data	Provide controlled resource update interfaces
Unauthorized access to patient/surgery information	Authentication and RBAC
Database/system failure	Regular backups and recovery mechanisms
Network failure	Retry mechanisms and graceful error handling

3.8 Traceability to Requirements

The architecture should directly map to requirements from the SRS.

SRS Requirement	Architectural Component
OTMS-F-001: User authentication	Authentication & RBAC Service
OTMS-F-002: Manage surgery records	Surgery Management Service
OTMS-F-003: Manage theatre slots	Scheduling Service
OTMS-F-004: Detect theatre conflicts	Conflict Detection Service
OTMS-F-005: Check doctor availability	Resource Management Service
OTMS-F-006: Check nurse availability	Resource Management Service
OTMS-F-007: Check equipment availability	Resource Management Service

OTMS-F-008: Prioritize emergency surgeries	Emergency Prioritization Service
OTMS-F-009: Reschedule affected surgeries	Scheduling Service
OTMS-F-010: Send schedule notifications	Notification Service
OTMS-F-011: Generate utilization reports	Reporting / Management Module

---

3.9 Security Architecture

The system uses role-based access control (RBAC) to ensure that users can access only the functionality and information appropriate to their roles.

Threat Modeling

A STRIDE-based approach is used:

Threat	Example	Mitigation
Spoofing	Unauthorized user pretending to be a doctor	Authentication + MFA where applicable
Tampering	Modification of surgery schedule	Authorization + audit logs
Repudiation	User denying a schedule change	Timestamped audit logs
Information Disclosure	Unauthorized access to patient data	Encryption + RBAC
Denial of Service	Excessive requests affecting scheduling	Rate limiting + monitoring

Elevation of Privilege	Nurse accessing administrator functions	Role-based authorization

---

4. Design

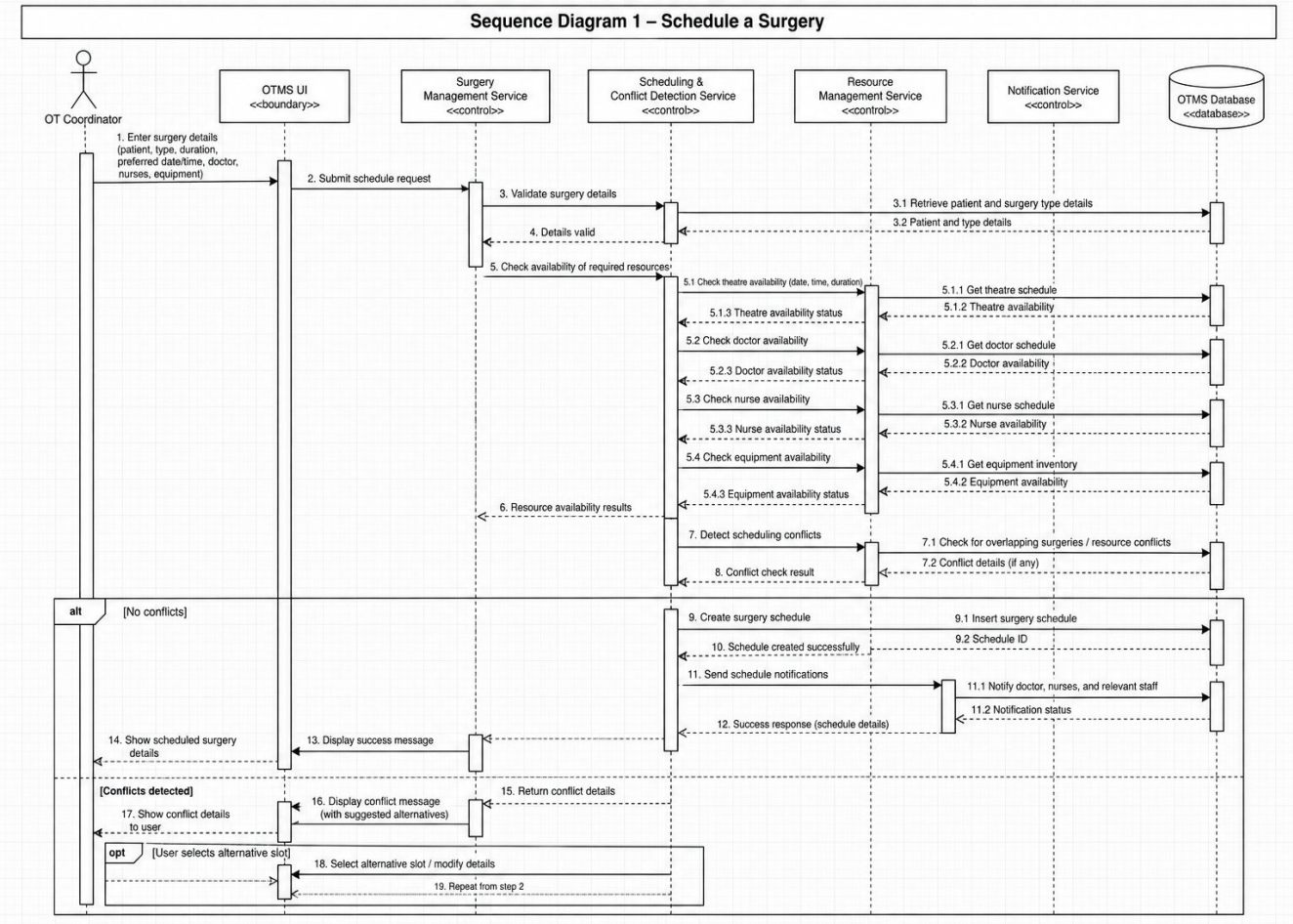
4.1 Design Overview

OTMS is designed using modular, layered components to separate user interaction, scheduling logic, resource management, security, and data storage.

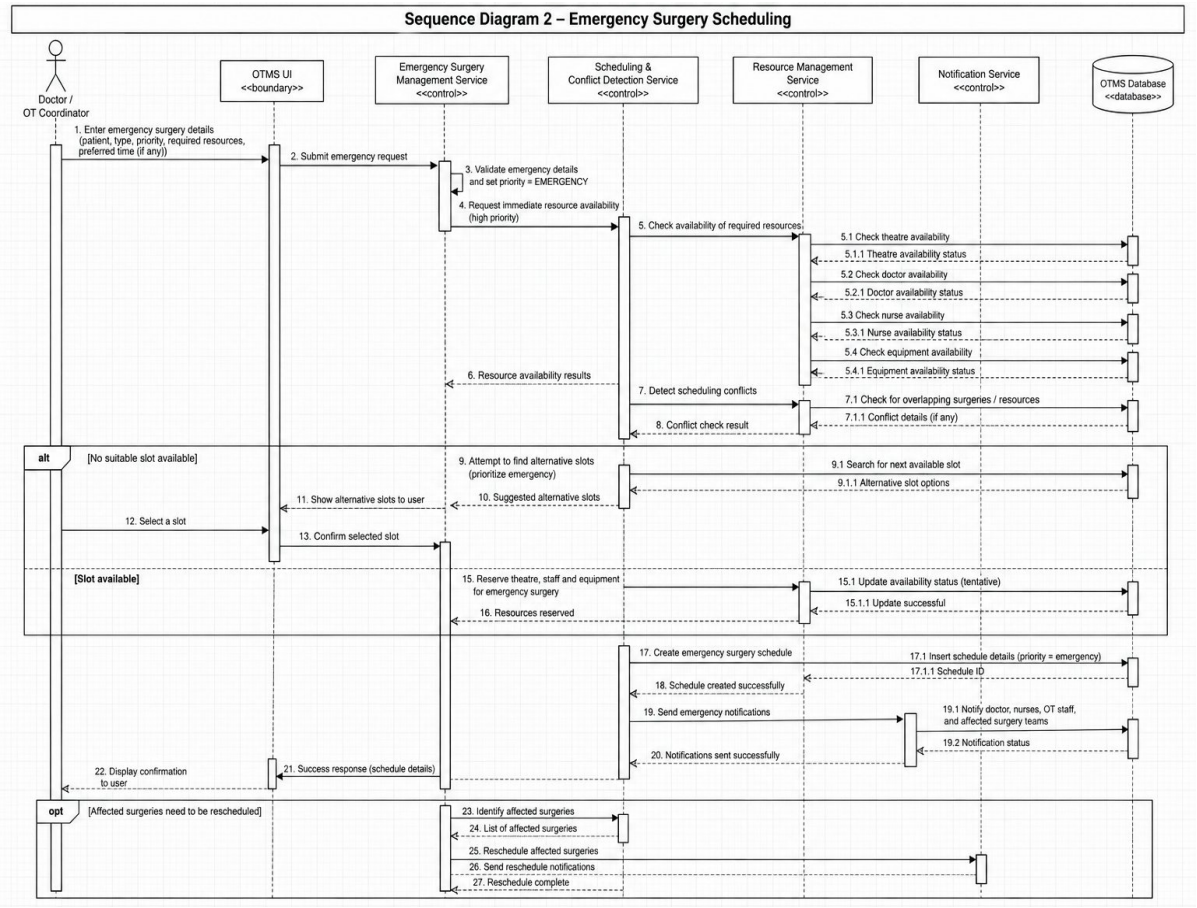
The central design focus is the scheduling engine, which validates theatre, doctor, nurse, and equipment availability before confirming a surgery. Emergency cases are assigned higher priority and may trigger controlled rescheduling of lower-priority surgeries.

4.2 UML Sequence Diagrams

Sequence Diagram 1 — Schedule a Surgery



Sequence Diagram 2 — Emergency Surgery Scheduling



4.3 API Design

API 1 — Schedule Surgery

Endpoint: /api/surgeries/schedule
Method: POST

Request:

```
{
  "surgeryId": "S1001",
  "doctorId": "D101",
  "nurseIds": ["N201", "N202"],
  "theatreId": "OT03",
  "equipmentIds": ["EQ12", "EQ18"],
  "startTime": "10:00",
  "endTime": "12:00"
```

}

Response:


```
{
  "status": "SCHEDULED",
  "surgeryId": "S1001",
  "theatreId": "OT03"
}
```


Errors:

- 400 Invalid scheduling details
- 409 Scheduling conflict
- 404 Resource not found

API 2 — Check Resource Availability

Endpoint: /api/resources/availability
Method: GET

Request Parameters:

theatreId
startTime
endTime

Response:


```
{
  "theatreAvailable": true,
  "doctorAvailable": true,
  "nurseAvailable": true,
  "equipmentAvailable": false
}
```


Errors:

- 400 Invalid time range
- 404 Resource not found

API 3 — Emergency Surgery

Endpoint: /api/surgeries/emergency
Method: POST

Request:

```
{
  "surgeryId": "S1005",
  "priority": "EMERGENCY",
  "requiredDuration": 120
}
```

Response:

```
{
  "status": "PRIORITIZED",
  "affectedSchedules": ["S1002", "S1003"]
}
```

Errors:

- 404 Surgery not found
- 409 No suitable theatre/resources available

4.4 Error Handling, Logging & Monitoring

Error Handling

The system uses standardized error responses for:

- Invalid user input
- Scheduling conflicts
- Unavailable theatres
- Unavailable doctors or nurses
- Equipment conflicts
- Database failures
- Unauthorized access

Sensitive patient information should not be included in error messages.

Logging

The system maintains logs for:

- User login and logout
- Surgery creation/modification
- Schedule changes
- Resource allocation
- Emergency prioritization
- Conflict detection
- System errors

Sensitive patient information and authentication credentials must not be stored unnecessarily in application logs.

Monitoring

The system monitors:

- Scheduling failure rate
- Number of active surgeries
- Theatre utilization
- Resource utilization
- Conflict frequency
- Emergency scheduling events
- System availability
- API response time

4.5 UX Design

The OTMS interface should provide a clear dashboard for each user role.

OT Coordinator

- View daily/weekly theatre schedule
- Schedule and reschedule surgeries
- View conflicts
- View theatre availability
- Prioritize emergency cases

Doctor

- View assigned surgeries
- View theatre and time allocation
- View required resources
- Receive schedule-change notifications

Nurse

- View assigned surgeries
- View duty schedule
- View required equipment/resources

Technician

- View equipment status
- Mark equipment as available/unavailable
- View maintenance requirements

Administrator

- Manage users and roles
- Manage theatres and equipment
- View utilization reports
- Monitor system activity

The UI should use clear status indicators, readable text, consistent navigation, and responsive layouts to reduce scheduling errors.

4.6 Open Issues & Next Steps

Potential future enhancements include:

- AI-assisted surgery scheduling and resource optimization
- Predictive theatre utilization analysis
- Mobile application for doctors and nurses
- Automated SMS/email notifications
- Integration with electronic health record systems
- Advanced analytics for theatre utilization
- Automatic prediction of surgery duration

- Integration with hospital inventory management
- Biometric authentication for privileged users

---

5. Appendices

5.1 Glossary

Term	Definition
OTMS	Operation Theatre Management System
OT	Operation Theatre
RBAC	Role-Based Access Control
API	Application Programming Interface
UI	User Interface
ADR	Architecture Decision Record
TLS	Transport Layer Security
STRIDE	Security threat-modeling methodology

5.2 References

- IEEE 42010 – Architecture Description
- OWASP Application Security Guidelines
- NIST Security and Privacy Guidelines
- Project Software Requirements Specification (SRS)
- Project Software Test Plan (STP)

5.3 Tools

- PlantUML – UML diagrams
- draw.io – Architecture and UML diagrams

- Postman – API testing
- Git/GitHub – Version control
- MySQL/PostgreSQL – Database
